

VOI2024 – Daknong

Hướng dẫn giải bài tập

19-20/12/2023

Ngày 20 tháng 12 năm 2023

Bài 1. NOEL

Bài 2. Cho cừu vào chuồng – BARNSHEEP

Bài 3. Ba chú chó con – TRIDOGS

Bài 1. Cá mập cắn cáp – SHARK

Bài 2. RECTAREA

Bài 3. Tập lái xe – F1TRAINING

Bài 1. Cặp câu ý nghĩa – MEANING

Bài 2. Bắn bóng bay – BALLOONS

Bài 3. Rèm sân khấu – CURTAINS

Mùa giáng sinh — Noel

Chính quyền thành phố Dresden đã mua N cây thông trang trí trên con đường lớn nhất của thành phố này. Họ sẽ tiến hành đặt N cây thông thành 2 hàng ở 2 bên của con đường. Một cách đặt sẽ khiến con đường lộng lẫy nếu như:

- ▶ Mỗi hàng đều phải có cây thông.
- ▶ Chênh lệch chiều cao giữa 2 cây thông liên tiếp trong cùng một hàng là như nhau và chiều cao của các cây sắp xếp theo thứ tự không giảm.

Yêu cầu: Hãy đề ra một phương án đặt các cây thông sao cho con đường trở nên lộng lẫy. Hoặc nếu không thể, hãy báo cáo lên chính quyền.

Subtask 1 ($N \leq 15$): $O(2^N)$

Thực hiện duyệt các dãy nhị phân, các vị trí có số 0 thì cây thông ở các vị trí đó sẽ cùng một hàng, tương tự với các vị trí có số 1. Sau khi có được các cây thông ở hai hàng, thực hiện sắp xếp không giảm và kiểm tra.

Subtask 2 ($N \leq 300$): $O(N^3)$

Thực hiện sắp xếp dãy $h_1, h_2, \dots, h_N$ không giảm. Nhận thấy mỗi hàng cây đều phải là một dãy cấp số cộng. Ta có các trường hợp sau:

- ▶ Với $N \leq 4$ thì ta chia các cây vào 2 hàng sao cho mỗi hàng không quá 2 cây thông.
- ▶ Với $N > 4$ thì luôn tồn tại một hàng có nhiều hơn 2 cây thông, giả sử đó là hàng bên trái. Bởi vì hàng bên trái là một dãy cấp số cộng nên ta chỉ cần quan tâm 2 số đầu tiên của hàng đó. Ta duyệt 2 giá trị $h_i, h_j (i < j)$ là hai cây đầu tiên của hàng bên trái. Nếu $N - 2$ cây còn lại có thể tạo ra một dãy cấp số cộng thì thỏa mãn, nếu không thì ta thêm một cây có giá trị bằng $2 * h_j - h_i$ vào hàng bên trái và kiểm tra $N - 3$ cây còn lại có thỏa mãn hay không. Cứ như vậy, ta cứ tiến hành thêm 1 cây mới (nếu có tồn tại) vào hàng bên trái và kiểm tra các cây còn lại có thể tạo ra dãy cấp số cộng hay không.

Subtask 3 ($N \leq 10^5$ và luôn tồn tại một phương án sao cho số lượng cây thông ở 2 hàng là như nhau)

Thực hiện sắp xếp dãy $h_1, h_2, \dots, h_N$ không giảm. Ta xét cách giải quyết với $N > 4$ và chỉ có 3 trường hợp sau:

- ▶ Hai số đầu tiên của hàng bên trái là h_1, h_2 .
- ▶ Hai số đầu tiên của hàng bên trái là h_1, h_3 .
- ▶ Hai số đầu tiên của hàng bên trái là h_2, h_3 .

Với mỗi trường hợp, ta sẽ tạo một dãy cấp số cộng gồm $N/2$ phần tử, và kiểm tra $N/2$ phần tử còn lại có tạo thành một dãy cấp số cộng hay không.

ĐPT: $O(N * 3)$.

Subtask 4 ($N \leq 10^5$): $O(N * 3 * \log N)$

Thực hiện sắp xếp dãy $h_1, h_2, \dots, h_N$ không giảm. Ta thực hiện các trường hợp như ở Subtask 3, vấn đề của ta là phải kiểm tra rằng sau khi thêm 1 cây mới vào hàng bên trái thì các cây còn lại có thể tạo thành một dãy cấp số cộng hay không. Để giải quyết vấn đề này, ta sử dụng 2 multiset S và T . Multiset S để lưu chiều cao h_i của các cây còn lại, multiset T lưu chênh lệch chiều cao giữa 2 cây liên tiếp trong S .

Khi thêm một cây mới có chiều cao y vào hàng bên trái thì:

- ▶ Xóa 1 giá trị y trong multiset S .
- ▶ Đặt x là số liền trước y , z là số liền sau y trong multiset S sau đó tiến hành xóa 2 giá trị $y - x$ và $z - y$ trong T , đồng thời thêm giá trị $z - x$ vào multiset T . Nếu tất cả các giá trị trong T bằng nhau (giá trị bé nhất bằng giá trị lớn nhất) chứng tỏ các cây còn lại có thể tạo thành 1 dãy cấp số cộng.

Bài 1. NOEL

Bài 2. Cho cừu vào chuồng – BARNSHEEP

Bài 3. Ba chú chó con – TRIDOGS

Bài 1. Cá mập cắn cáp – SHARK

Bài 2. RECTAREA

Bài 3. Tập lái xe – F1TRAINING

Bài 1. Cặp câu ý nghĩa – MEANING

Bài 2. Bắn bóng bay – BALLOONS

Bài 3. Rèm sân khấu – CURTAINS

Cho cừu vào chuồng – BARNSHEEP

- ▶ Nông trại của Phương có N con cừu và N căn chuồng cừu. Phương thu được 2 dãy số nguyên $s_1, s_2, \dots, s_N$ là kích thước từng con cừu và $t_1, t_2, \dots, t_N$ là kích thước từng căn chuồng. Con cừu thứ i có thể ở vào căn chuồng thứ j nếu $s_i \leq t_j$ và mỗi chuồng cũng chỉ chứa tối đa được một con cừu.
- ▶ Một cách phân bố các con cừu vào chuồng được gọi là "cực đại" nếu như trong cách phân bố đó ta không thể thêm bất kỳ con cừu nào vào một chuồng khác được nữa.
- ▶ **Yêu cầu:** Hãy giúp Phương xác định được có bao nhiêu cách phân bố "cực đại" khác nhau.

Subtask 1 ($N \leq 8$): $O(N! * N)$

Duyệt hoán vị tất cả các trường hợp phân bố cừu vào các chuồng.

Subtask 2 ($N \leq 50$): $O(N^3)$

Ta sắp xếp kích thước các con cừu cùng với các chuồng theo thứ tự không giảm.

Thực hiện duyệt con cừu S_x là con cừu có kích thước nhỏ nhất và không được phân bổ vào bất kỳ chuồng nào. Ta có:

- ▶ Tất cả những con cừu S_y với $y < x$ bắt buộc phải có chuồng.
- ▶ Tất cả những chuồng có $T_y \geq S_x$ phải có chứa cừu.

Subtask 2 ($N \leq 50$): $O(N^3)$

Duyệt theo thứ tự kích thước không giảm của cả cừu và chuồng, gọi $dp[i, c]$ là số cách phân bố để con cừu S_x là con cừu nhỏ nhất không có chuồng khi xét đến phần tử thứ i (có thể là cừu hoặc chuồng) và có c con cừu ta cho vào chuồng nhưng hiện chưa có chuồng, xem như chúng đang ở trong một hàng đợi.

Khởi tạo $dp[0, 0] = 1$

- ▶ Khi phần tử thứ i là một con cừu:
 - ▶ Nếu $i < x$ tức là con cừu bắt buộc phải có chuồng:
 $dp[i, c] += dp[i - 1, c - 1]$.
 - ▶ Nếu $i \geq x$ thì con cừu này có thể được phân vào chuồng hoặc không: $dp[i, c] += dp[i - 1, c - 1] + dp[i, c]$.
- ▶ Khi phần tử thứ i là một cái chuồng:
 - ▶ Nếu $T_i \geq S_x$ thì chuồng bắt buộc chứa cừu:
 $dp[i, c] += dp[i - 1, c + 1] * (c + 1)$.
 - ▶ Nếu $T_i < S_x$ thì chuồng không bắt buộc chứa cừu:
 $dp[i, c] += dp[i - 1, c] + dp[i - 1, c + 1] * (c + 1)$.

Subtask 3 ($N \leq 3000$): $O(N^2)$

Ta không cần thực hiện việc duyệt con cừu S_x như subtask 2, thay vào đó ta có thêm 1 chiều trong QHĐ với hai giá trị 0/1 cho biết:

- ▶ 0 là chưa chọn ra con cừu nhỏ nhất không có chuồng.
- ▶ 1 là đã chọn ra con cừu nhỏ nhất không có chuồng.

Duyệt theo thứ tự kích thước không giảm của cả cừu và chuồng.

- ▶ Khi phần tử thứ i là cừu:
 - ▶ $dp[i, c, l] + = dp[i, c - 1, l]$: con cừu này được phân vào chuồng (vào hàng đợi).
 - ▶ $dp[i, c, 1] + = dp[i, c, l]$: con cừu này không được phân vào chuồng.
- ▶ Khi phần tử thứ i là chuồng:
 - ▶ $dp[i, c, l] + = dp[i - 1, c + 1, l] * (c + 1)$.
 - ▶ $dp[i, c, 0] + = dp[i - 1, c, 0]$.

Bài 1. NOEL

Bài 2. Cho cừu vào chuồng – BARNSHEEP

Bài 3. Ba chú chó con – TRIDOGS

Bài 1. Cá mập cắn cáp – SHARK

Bài 2. RECTAREA

Bài 3. Tập lái xe – F1TRAINING

Bài 1. Cặp câu ý nghĩa – MEANING

Bài 2. Bắn bóng bay – BALLOONS

Bài 3. Rèm sân khấu – CURTAINS

Ba chú chó con – TRIDOGS

- ▶ Có N chú chó được chọn trong khóa huấn luyện, đứng nghiêm xếp thành một hàng ngang. Chú chó thứ i có màu lông là a_i . Huấn luyện viên quyết định, trong bài tập lần này sẽ chọn ra một nhóm gồm các chú cún đứng liên tiếp nhau và nhóm phải có tối thiểu ba thành viên.
- ▶ Một cách lập nhóm tương đương với một cách chọn bộ ba chỉ số (i, j, k) thỏa mãn $1 \leq i < j < k \leq N$, các chú chó thứ $i, i+1, \dots, k$ sẽ là thành viên của nhóm và ba chú chó thứ i, j, k sẽ được chọn làm đội trưởng, mỗi chú chó được chọn làm đội trưởng thì màu lông phải khác biệt với tất cả thành viên còn lại trong nhóm (kể cả đối với đội trưởng khác).
- ▶ **Yêu cầu:** Hãy giúp huấn luyện viên tính xem có bao nhiêu cách để lập ra một nhóm thỏa mãn các tiêu chí trên.

Subtask 2 ($N \leq 500$): $O(N^3)$

Thực hiện duyệt các 3 chỉ số i, j, k và kết hợp dùng mảng đánh dấu. Trước hết đánh dấu các phần tử $a_i, a_{i+1}, \dots, a_j$. Sau đó trong quá trình duyệt chỉ số k , ta đánh dấu a_k . Nếu a_k chưa xuất hiện trước đó thì tăng kết quả lên 1.

Subtask 3 ($N \leq 5000$): $O(N^2)$

Thực hiện duyệt các chỉ số i, k ($k = 1 \leftarrow N, i = k \leftarrow 1$) và duy trì 1 biến *unique* là số lượng chỉ số $j \in [i + 1, k - 1]$ thỏa mãn a_j xuất hiện duy nhất trong đoạn $a_i, a_{i+1}, \dots, a_k$.

Trong quá trình duyệt chỉ số i ($i = k \leftarrow 1$), ta sử dụng mảng đánh dấu để đếm số lần xuất hiện của các giá trị.

- ▶ Nếu $a_i = a_k$ thì ta dừng lại.
- ▶ Nếu a_i xuất hiện duy nhất trong đoạn $a_i, a_{i+1}, \dots, a_k$ thì $Ans+ = unique$ và $unique+ = 1$.
- ▶ Nếu a_i xuất hiện lần thứ hai trong đoạn $a_i, a_{i+1}, \dots, a_k$ thì $unique- = 1$.

Subtask 4 ($N \leq 2 * 10^5$): $O(N * \log N)$

Ta sẽ duy trì các thông tin sau:

- ▶ $pre[i] = j$ là vị trí lớn nhất thỏa mãn $j < i$ và $a_j = a_i$.
- ▶ $active_k[i] = 1$ nếu a_i xuất hiện duy nhất trong đoạn $a_i, a_{i+1}, \dots, a_k$ và $= 0$ nếu ngược lại.
- ▶ $unique_k[i] =$ số lượng chỉ số $j \in [i + 1, k - 1]$ thỏa mãn a_j xuất hiện duy nhất trong đoạn $a_i, a_{i+1}, \dots, a_k$.
- ▶ $val_k[i] = active_k[i] * unique_k[i]$ là số lượng bộ ba chỉ số (i, j, k) thỏa mãn.

Subtask 4 ($N \leq 2 * 10^5$): $O(N * \log N)$

Thực hiện duyệt k ($k = 1 \leftarrow N$), quá trình chuyển đổi từ $k - 1$ sang k thì các giá trị sau sẽ thay đổi:

- ▶ $active_k[k] = 1$.
- ▶ $active_k[pre[k]] = 0$.
- ▶ $unique_k[pre[k - 1] + 1, \dots, k - 2]$ tăng lên 1. Ta hiểu rằng lúc này a_{k-1} là giá trị xuất hiện duy nhất trong các đoạn $a_i, a_{i+1}, \dots, a_k$ với $i \in [pre[k - 1] + 1, \dots, k - 2]$.
- ▶ $unique_{k+1}[pre[pre[k]] + 1, \dots, pre[k] - 1]$ giảm đi 1. Do khi duyệt sang a_k thì giá trị $a_{pre[k]}$ không còn là giá trị xuất hiện duy nhất trong các đoạn $a_i, a_{i+1}, \dots, a_k$ với $i \in [pre[pre[k]] + 1, \dots, pre[k] - 1]$.
- ▶ $Ans+ = \sum_{i=pre[k]+1}^{k-2} val_k[i]$.

Để cập nhật các giá trị như trên và lấy tổng để tính vào kết quả thì ta sử dụng segment tree lazy.

Bài 1. NOEL

Bài 2. Cho cừu vào chuồng – BARNSHEEP

Bài 3. Ba chú chó con – TRIDOGS

Bài 1. Cá mập cắn cáp – SHARK

Bài 2. RECTAREA

Bài 3. Tập lái xe – F1TRAINING

Bài 1. Cặp câu ý nghĩa – MEANING

Bài 2. Bắn bóng bay – BALLOONS

Bài 3. Rèm sân khấu – CURTAINS

Cá mập cắn cáp – SHARK

- ▶ Có N đảo quốc và hệ thống dây cáp quang nối 2 đảo kề nhau, hệ thống dây cáp này đã tạo thành một đa giác đều với mỗi đỉnh là vị trí một quốc gia. Người ta muốn xây dựng thêm các đường dây cáp quang mới dưới vùng biển được giới hạn bởi các dây cáp ban đầu. Các quốc gia sẽ cùng xây dựng thêm $N - 3$ tuyến cáp mới, đảm bảo không có hai tuyến nào giao nhau (trừ hai đầu mút). Tuyến cáp thứ k nối hai quốc gia phân biệt i_k và j_k .
- ▶ Một cỗ máy cá mập có thể cắn đứt mọi dây cáp quang trên đường đi trên một đường thẳng của nó. Hãy tính toán xem số lượng dây cáp tối đa có thể bị cắn đứt là bao nhiêu.

Subtask 1 (10%): $i_k = 1, \forall k$

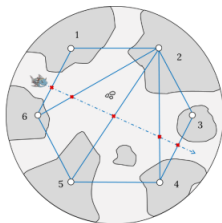
Khi tất cả đường chéo được kẻ từ 1 đỉnh. Ta hoàn toàn có thể kẻ một đường thẳng đi qua tất cả các đường chéo, ngoài ra còn đi qua hai cạnh của đa giác. Vì vậy, kết quả sẽ là $N - 1$.

Subtask 2,3 (20%): $N \leq 500$

- ▶ Đưa đa giác lên hệ trục Oxy . Đặt gốc O là tâm, vẽ một đường tròn bán kính R . Chọn N điểm cách đều nhau trên đường tròn đó, N điểm đó sẽ là N đỉnh của đa giác.
- ▶ Duyệt từng cặp cạnh của đa giác, vẽ một đoạn thẳng đi từ một điểm của cạnh này tới một điểm của cạnh kia. Đếm số cạnh giao với đường thẳng vừa kẻ.

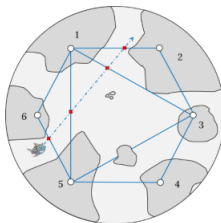
Giải thích

ví dụ 1



Nếu cỗ máy đi theo đường đi trên, số dây cáp tới đa có thể bị cắt đứt là 5.

ví dụ 2



Nếu cỗ máy đi theo đường đi trên, số dây cáp tới đa có thể bị cắt đứt là 4.

Subtask 4 (30%): $N \leq 5000$

- ▶ Ý tưởng: xem từng tam giác được tạo là mỗi đỉnh, với hai tam giác có chung cạnh sẽ kề nhau. Bài toán sẽ trở thành tìm đường đi dài nhất trên cây.
- ▶ Xét từng đỉnh của đa giác. Tại đỉnh thứ i , gọi S_i là tập các đỉnh thuộc đa giác sao cho tồn tại một đường chéo kết nối với đỉnh i . Sắp xếp các đỉnh theo chiều kim đồng hồ. Với đỉnh $S[j]$, ba đỉnh $i, S[j], S[j + 1]$ tạo thành một tam giác. Ta sử dụng map đánh số cho tam giác này. Ngoài ra, tam giác $(i, S[j], S[j + 1])$ sẽ có một cạnh chung với tam giác $(i, S[j + 1], S[j + 2])$.
- ▶ Sau khi dựng đồ thị (dạng cây), từ mỗi đỉnh, BFS hoặc DFS tìm đường đi dài nhất.

Subtask 5 (20%): $N \leq 10^5$

- ▶ Dựng đồ thị tương tự ở Subtask 4, nhưng tối ưu phần tìm đường đi dài nhất.
- ▶ Coi đỉnh 1 là gốc.
- ▶ Gọi $f[i]$ là đường đi dài nhất từ đỉnh i trong cây con gốc i . Có: $f[i] = \max(f[j] + 1)$ với j là con của i . Ngoài ra, khi ta kết nối đường đi dài nhất trong cây con gốc i , lên i , và xuống đường đi dài nhất trong cây con gốc i , ta sẽ được đường đi dài nhất trong cây con gốc i .

Bài 1. NOEL

Bài 2. Cho cừu vào chuồng – BARNSHEEP

Bài 3. Ba chú chó con – TRIDOGS

Bài 1. Cá mập cắn cáp – SHARK

Bài 2. RECTAREA

Bài 3. Tập lái xe – F1TRAINING

Bài 1. Cặp câu ý nghĩa – MEANING

Bài 2. Bắn bóng bay – BALLOONS

Bài 3. Rèm sân khấu – CURTAINS

RECTAREA

<https://oj.vnoi.info/problem/area>

Tính diện tích che phủ bởi N hình chữ nhật, biết rằng N hình chữ nhật này song song với 2 trục Ox và Oy .

Subtask 1: $N \leq 8$

Subtask 2: $N \leq 20$

Đếm bao hàm loại trừ các phần hình chữ nhật giao nhau.

DPT: $O(N \times 2^N)$

Subtask 3: $N \leq 100$, các toạ độ đều ≤ 100

Với mỗi hình chữ nhật, tăng giá trị các ô nằm trong hình chữ nhật đó lên 1 và tính tổng các ô có giá trị > 0 .

ĐPT: $O(N \times Q^2)$ với $Q = 100$

Subtask 4: $N \leq 1000$

- ▶ Sắp xếp các cạnh song song với trục tung tăng dần. Có 2 loại cạnh:
 - ▶ Cạnh đóng (+1): là cạnh bên trái của hình chữ nhật tương ứng
 - ▶ Cạnh mở (-1): là cạnh bên phải của hình chữ nhật tương ứng
- ▶ Sử dụng kỹ thuật sweep line quét theo các cạnh theo thứ tự sắp xếp đó, cộng hoặc trừ (tùy theo thứ tự cạnh đóng mở) phần phủ mặt phẳng giữa hai đường thẳng song song đi qua 2 cạnh liên tiếp đang xét
- ▶ ĐPT: $O(n^2)$

Subtask 5: $N \leq 10^5$, các toạ độ đều ≤ 5000

- ▶ Sử dụng mảng cộng dồn để tính $F(x, y)$ là số lượng các hình chữ nhật bao phủ ô (x, y)
- ▶ Với mỗi hình chữ nhật (i, j, u, v) góc trái trên (i, j) và góc phải dưới (u, v) , đặt:
 $F(i, j)++; F(i, v+1)--; F(u+1, y)--; F(u+1, v+1)++$
- ▶ Duyệt dọc theo các hàng để cộng dồn $F(i, j)+ = F(i-1, j)$
- ▶ Duyệt dọc theo các cột để cộng dồn $F(i, j)+ = F(i, j-1)$
- ▶ Cuối cùng dựa vào giá trị trong mảng F sau khi chuẩn bị để tính diện tích bao phủ
- ▶ ĐPT $O(N + Q^2)$ với $Q = 5000$

Subtask 6: $N \leq 10^5$, các toạ độ đều $\leq 10^5$

- ▶ Sắp xếp các cạnh song song với trục tung tăng dần
- ▶ Sử dụng kỹ thuật sweep line quét theo các cạnh theo thứ tự sắp xếp đó và update dần lên cây IT quản lý theo trục Oy . Khi quét đến cạnh $[\ell, r]$, gọi h là đường thẳng đi qua cạnh này:
 - ▶ Xét hai đường song song liên tiếp h và h' tiếp theo, gọi là k là chiều dài đoạn thuộc h mà được phủ bởi các hình chữ nhật đã được update lên cây IT, phần diện tích được tính thêm vào đến đường h' sẽ là $k \times (h' - h)$
 - ▶ Nếu gặp đoạn từ ℓ đến r thì sẽ update lên cây IT tăng đoạn $[\ell, r]$ lên 1 đơn vị.
- ▶ DPT: $O(N \log(N) + N \log Q)$ với $Q = 10^5$

Subtask 7: $N \leq 10^5$, các toạ độ đều $\leq 10^9$

- ▶ Giống Subtask 6 nhưng phải cài đặt bằng kỹ thuật cây IT động
- ▶ ĐPT: $O(N \log(N) + N \log Q)$ với $Q = 10^9$

Bài 1. NOEL

Bài 2. Cho cừu vào chuồng – BARNSHEEP

Bài 3. Ba chú chó con – TRIDOGS

Bài 1. Cá mập cắn cáp – SHARK

Bài 2. RECTAREA

Bài 3. Tập lái xe – F1TRAINING

Bài 1. Cặp câu ý nghĩa – MEANING

Bài 2. Bắn bóng bay – BALLOONS

Bài 3. Rèm sân khấu – CURTAINS

Tập đua xe – F1TRAINING

- ▶ Bắt đầu khi xe của Hoàng xuất phát tại điểm 0 với mục tiêu chạy đến đích điểm S nhanh nhất có thể.
- ▶ Vận tốc ban đầu của xe là v_0 , sau đó HLV cung cấp cho Hoàng N phương pháp để thay đổi vận tốc một cách tối ưu, và yêu cầu Hoàng chỉ được thay đổi vận tốc xe ở trên các điểm nguyên mà thôi. Phương pháp thứ i gồm hai số nguyên c_i và v_i , nghĩa là, xe của Hoàng sẽ lùi lại c_i đơn vị trên quãng đường, sau đó vận tốc của xe sẽ thay đổi thành v_i . Thời gian xe lùi về vị trí đó coi như không đáng kể.
- ▶ Biết rằng Hoàng có thể sử dụng một phương pháp thay đổi vận tốc nhiều lần. Với những phương pháp thay đổi vận tốc mà HLV đưa ra, bạn hãy thử giúp Hoàng tính toán xem thời điểm sớm nhất để đi tới đích S là bao nhiêu.
- ▶ Lưu ý, trong mỗi chặng Hoàng di chuyển, bạn cần tính thời điểm sớm nhất là số nguyên (làm tròn lên).

Subtask 1 ($N, S \leq 100$): $O(N^2.S.\log(N.S))$)

Ta gọi $f[i, p]$ là số đơn vị thời gian ít nhất cần phải bỏ ra để đến được vị trí p , trong đó i là phương pháp cuối cùng áp dụng để đổi vận tốc. Có hai trường hợp:

- ▶ Vẫn sử dụng phương pháp cuối cùng để đi tiếp: Khi đó $f[i, p]$ sẽ cập nhật cho $f[i, p + v_i]$ và thêm 1 đơn vị thời gian.
- ▶ Nếu chọn một phương pháp đổi vận tốc khác là j : Khi đó $f[i, p]$ sẽ cập nhật cho $f[j, p - c_j]$ và không cần thêm đơn vị thời gian nào cả.

Do tính không thứ tự của liên kết giữa các hàm nên chúng ta sẽ không thể QHĐ, thay vào đó ta có thể dùng Dijkstra để tính kết quả tối ưu là $f[i, S']$ với $S' \geq S$. Xây dựng đồ thị khá đơn giản với một cặp (i, p) là một đỉnh trên đồ thị.

Subtask 2 ($N \leq 1000$): $O(N^2)$

Chúng ta sẽ giải quyết bài toán theo một hướng khác:

- ▶ Sort các lựa chọn thay đổi tốc độ theo tốc độ giảm dần. Rõ ràng, chúng ta sẽ không thay đổi về vận tốc bé hơn. Vậy sort theo thứ tự giảm dần để phục vụ cho tối ưu sau này.
- ▶ Có một nhận xét là nếu chúng ta sử dụng lựa chọn thay đổi vận tốc v_i , thì chúng ta sẽ cố gắng sử dụng chúng càng sớm càng tốt (vì trước đó chúng ta đang ở vận tốc thấp hơn, do đó ngay khi ở vị trí lớn hơn hoặc bằng c_i , chúng ta sẽ sử dụng phương pháp thay đổi vận tốc thay đổi này).
- ▶ Sử dụng QHD: với $f[i]$ là thời điểm ngắn nhất có thể để bạn có thể sử dụng phương pháp thay đổi vận tốc thứ i và $g[i]$ là vị trí xe của bạn sau khi sử dụng phương pháp i (tại thời điểm $f[i]$). Chúng ta có thể dễ dàng tính được $f[i]$ và $g[i]$ bằng cách xét phương pháp thay đổi vận tốc dùng ngay trước phương pháp i .

Subtask 3 ($N \leq 200000$; $S \leq 10^{16}$; $1 \leq c_i, v_i \leq 10^9$):
 $O(N \cdot \log^2 N)$

Tiếp tục hướng giải quyết như subtask 2:

- ▶ Sau khi sử dụng phương pháp i , mỗi giây xe của bạn sẽ đi được v_i đơn vị. Do đó, tại các thời điểm x ($x \leq f[i]$), xe của bạn sẽ ở vị trí $g[i] + v[i] * (x - f[i])$.
- ▶ Thay vào đó, ta sẽ sử dụng Interval Tree trên tập đoạn thẳng để giải quyết bài toán này, tham khảo wiki:
<https://vnoi.info/wiki/algo/data-structures/interval-tree-tap-doan-thang.md>

Subtask 3 ($N \leq 200000$; $S \leq 10^{16}$; $1 \leq c_i, v_i \leq 10^9$):
 $O(N \cdot \log^2 N)$

- ▶ Đi vào chi tiết, xét biểu thức trên $(g[i] + v[i] * (x - f[i]))$, rõ ràng chúng ta có thể đại diện biểu thức dưới dạng một đường thẳng $a.x + b$, với $a = v[i]$ và $b = g[i] - v[i] * f[i]$.
- ▶ Khi tính $f[i]$ & $g[i]$, giả sử tất cả đoạn thẳng đại diện cho các phương pháp thay đổi vận tốc từ 1 đến $i - 1$ đã được tính và cập nhật trong cây IT. Khi đó, ta chắt nhai phân thời điểm sớm nhất có thể sử dụng phương pháp thay đổi vận tốc thứ i , sau đó kiểm tra xem tại thời điểm i thì có đoạn thẳng đại diện cho phương pháp thay đổi vận tốc nào có tung độ không nhỏ hơn c_i hay không? Nếu có, đồng nghĩa với việc có thể sử dụng tùy chọn này ngay sau tùy chọn của đoạn thẳng kia.
- ▶ Sau khi tính được $f[i], g[i]$, chúng ta có thể giả sử i là tùy chọn cuối cùng được dùng và tính được kết quả bài toán.

Bài 1. NOEL

Bài 2. Cho cừu vào chuồng – BARNSHEEP

Bài 3. Ba chú chó con – TRIDOGS

Bài 1. Cá mập cắn cáp – SHARK

Bài 2. RECTAREA

Bài 3. Tập lái xe – F1TRAINING

Bài 1. Cặp câu ý nghĩa – MEANING

Bài 2. Bắn bóng bay – BALLOONS

Bài 3. Rèm sân khấu – CURTAINS

Cặp câu ý nghĩa – MEANING

- ▶ Có N câu $S_1, S_2, \dots, S_N$ khác nhau, mỗi câu tương ứng với một xâu kí tự gồm các chữ cái tiếng anh in thường. Hai câu i, j ($i \neq j$) có ý nghĩa giống nhau nếu như S_i là một hoán vị của S_j .
- ▶ Romeo sẽ chọn ra tập hợp các câu từ N câu đã viết sao cho trong tập hợp đó có chính xác K cặp câu có ý nghĩa giống nhau.
- ▶ **Yêu cầu:** Hãy giúp Romeo xác định có bao nhiêu cách chọn tập hợp như vậy.

Subtask 1 ($N \leq 15$): $O(2^N * N)$

Thực hiện duyệt tất cả các tập hợp câu có thể chọn và kiểm tra xem có chính xác K cặp câu thỏa mãn hay không.

Subtask 2 ($0 \leq K \leq 3$): $O(4 * N)$

Thực hiện đưa các câu có ý nghĩa giống nhau vào cùng một nhóm. Giả sử thu được M nhóm như vậy, gọi $a_1, a_2, \dots, a_M$ là số lượng câu trong mỗi nhóm.

Ta giải quyết các trường hợp với các giá trị K khác nhau:

- ▶ $K = 0$ thì mỗi nhóm ta chọn không quá 1 câu trong đó để đưa vào tập hợp.
- ▶ $K = 1$ thì có một nhóm ta chọn 2 câu, các nhóm còn lại chọn không quá 1 câu để đưa vào tập hợp.
- ▶ $K = 2$ thì có hai nhóm mà ta chọn 2 câu ở mỗi nhóm, các nhóm còn lại chọn không quá 1 câu đưa vào tập hợp.
- ▶ $K = 3$ thì có ba nhóm mà ta chọn 2 câu ở mỗi nhóm, các nhóm còn lại chọn không quá 1 câu đưa vào tập hợp hoặc có một nhóm ta chọn 3 câu, các nhóm còn lại không quá 1 câu để đưa vào tập hợp.

Subtask 3 ($N, K \leq 2000$): $O(N * (N + K))$)

QHD: Gọi $dp[i, j]$ là số cách chọn tập hợp khi xét đến nhóm thứ i và đã có j cặp câu thỏa mãn. Ta có:

$$dp[i, j] = \sum_{t=0}^{t \leq a_i, t(t-1)/2 \leq k} \binom{a_i}{t} dp[i-1][j - t(t-1)/2]$$

với t là số lượng câu ta chọn vào tập từ nhóm $a[i]$, t câu này tạo ra $t * (t - 1)/2$ cặp.

Kết quả bài toán là $dp[M][K]$.

Do $a_1 + a_2 + \dots + a_M = N$ nên độ phức tạp bài toán là $O(N * (N + K))$.

Bài 1. NOEL

Bài 2. Cho cừu vào chuồng – BARNSHEEP

Bài 3. Ba chú chó con – TRIDOGS

Bài 1. Cá mập cắn cáp – SHARK

Bài 2. RECTAREA

Bài 3. Tập lái xe – F1TRAINING

Bài 1. Cặp câu ý nghĩa – MEANING

Bài 2. Bắn bóng bay – BALLOONS

Bài 3. Rèm sân khấu – CURTAINS

Bắn bóng bay – BALLOONS

- ▶ Người ta treo N quả bóng bay thành một hàng ngang, quả bóng bay thứ i có màu sắc là b_i .
- ▶ Người chơi cầm súng bắn các quả bóng bay, càng bắn được nhiều bóng thì phần thưởng càng lớn tuy nhiên phải tuân theo quy tắc sau: Gọi $i_1, i_2, \dots, i_m$ là chỉ số các quả bóng bay bị bắn theo thứ tự:
 - ▶ $i_1 < i_2 < \dots < i_m$.
 - ▶ Với $1 \leq j \leq m-2$ thì $b_{i_j} \neq b_{i_{j+1}}, b_{i_j} \neq b_{i_{j+2}}, b_{i_{j+1}} \neq b_{i_{j+2}}$.
- ▶ **Yêu cầu:** Hãy tính toán số lượng quả bóng bay tối đa có thể bắn và thỏa mãn quy tắc trò chơi.

Subtask 1 ($b_i \leq b_{i+1}$ với $1 \leq i < N$): $O(N)$

Kết quả bài toán là độ dài của dãy con tăng ngặt dài nhất có trong dãy $b_1, b_2, \dots, b_N$ hay có thể hiểu là số lượng giá trị khác nhau xuất hiện trong dãy $b_1, b_2, \dots, b_N$.

Subtask 2 ($N \leq 8$): $O(2^N)$

Duyệt tất cả các trường hợp chọn các quả bóng bay để bắn, sau đó kiểm tra xem từng trường hợp có vi phạm quy tắc hay không. Chọn ra trường hợp có số lượng bóng bay nhiều nhất thỏa mãn.

Subtask 3 ($\sum N \leq 500$): $O(N^3)$

Xem 1 dãy bóng bay ta bắn là một dãy con không liên tiếp của dãy $b_1, b_2, \dots, b_N$.

Gọi $dp[i, j]$ với $1 \leq i < j \leq N$ là độ dài dãy con dài nhất thỏa mãn đề bài với 2 quả bóng bay cuối cùng ta bắn lần lượt là b_i, b_j .

- ▶ Nếu $b_i = b_j$ thì $dp[i, j] = 0$.
- ▶ Ngược lại, $dp[i, j] = \max(2, \max_{1 \leq k < i} (dp[k, i] + 1))$ với k thỏa mãn $a_k \neq a_i$ và $a_k \neq a_j$.

$Ans = \max_{1 \leq i < j \leq N} dp[i, j]$.

Subtask 4 ($b_i \leq 3$ với $1 \leq i \leq N$): $O(N)$

Xét dãy con $b_{i_1}, b_{i_2}, \dots, b_{i_m}$ là một dãy bóng bay bị bắn thỏa mãn. Dễ thấy vì $b_i \leq 3$ nên:

- ▶ $\{b_{i_j}, b_{i_{j+1}}, b_{i_{j+2}}\}$ là một hoán vị của $\{1, 2, 3\}$.
- ▶ Và $b_{i_j} = b_{i_{j+3}}$.

Vậy ta xét 6 trường hợp khác nhau (mỗi trường hợp ứng với một dạng của hoán vị $\{1, 2, 3\}$), mỗi trường hợp ta thực hiện tham lam để lấy được dãy dài nhất có thể.

Subtask 5 ($b_i \leq 10$ với $1 \leq i \leq N$): $O(10^2 * N)$

Gọi $dp[x, y]$ là độ dài dãy con dài nhất thỏa mãn đề bài với 2 giá trị cuối cùng trong dãy lần lượt là x, y .

Mỗi khi duyệt đến giá trị b_i , ta cần tính toán và cập nhập giá trị $dp[x, b_i]$ với $x \neq b_i$.

- ▶ Nếu không tồn tại giá trị x phía trước b_i , ta cập nhập $dp[x, b_i] = 1$.
- ▶ Ngược lại, duyệt giá trị y thỏa mãn $y \neq x$ và $y \neq b_i$, cập nhập $dp[x, b_i] = \max(dp[x, b_i], dp[y, x] + 1)$.

$Ans = \max_{1 \leq x < y \leq 10} dp[x, y]$.

Subtask 6 ($\sum N \leq 10000$): $O(N^2)$)

- ▶ Ý tưởng của subtask 6 khá giống với subtask 5.
- ▶ Đầu tiên để giảm giá trị các số ta có thể tiến hành rời rạc chúng. Lúc này giá trị các số $b_1, b_2, \dots, b_N$ đều không vượt quá N .
- ▶ Theo subtask 5 ta có công thức:

$$dp[x, b_i] = \max(dp[x, b_i], dp[y, x] + 1), x \neq b_i, y \neq b_i, y \neq x.$$

- ▶ Nếu làm hết như subtask 5 thì ĐPT là $O(N^3)$. Tuy nhiên, ta thấy chỉ cần quan tâm 2 giá trị y_1, y_2 khác nhau thỏa mãn:
 - ▶ $y_1 \neq x, y_2 \neq x$.
 - ▶ $dp[y_1, x]$ đạt giá trị lớn nhất, $dp[y_2, x]$ đạt giá trị lớn thứ hai.
- ▶ Chắc chắn rằng ít nhất một trong hai giá trị y_1, y_2 sẽ khác với b_i .
- ▶ Vậy với mỗi $dp[y, x]$ ta lưu thêm 2 giá trị y_1, y_2 như trên và sau đó khi tính $dp[x, b_i]$ ta chỉ cần dùng đến $dp[y_1, x], dp[y_2, x]$.

Subtask 7 ($N \leq 2 * 10^5$): $O(N * (5^2 + \log N))$

Ta có các nhận xét sau:

- ▶ Xét tại số b_i , nếu phía trước b_i có một giá trị x ($x \neq b_i$) xuất hiện nhiều lần thì cách tối ưu nhất là ta chọn giá trị x có vị trí xuất hiện cuối cùng đứng phía trước b_i .
- ▶ Xét tối đa 5 giá trị x_1, x_2, x_3, x_4, x_5 khác nhau có vị trí xuất hiện cuối cùng bên phải nhất đứng trước b_i . Theo subtask 6, ta cần duyệt tất cả giá trị x để tính $dp[x, b_i]$, tuy nhiên điều đó không cần thiết, vì giá trị x phù hợp sẽ rơi vào 1 trong 5 giá trị x_1, x_2, x_3, x_4, x_5 .
- ▶ Chứng minh: nếu giá trị x đứng trước b_i trong dãy con mà $x \notin \{x_1, x_2, x_3, x_4, x_5\}$ thì ta có thể chèn thêm 1 giá trị $\in \{x_1, x_2, x_3, x_4, x_5\}$ vào giữa x và b_i , như thế dãy con mới tối ưu về độ dài.

Kết luận đối với mỗi số b_i , ta chỉ có tối đa 5 số (5 giá trị x) khác nhau cần xét để tiến hành quy hoạch động.

Bài 1. NOEL

Bài 2. Cho cừu vào chuồng – BARNSHEEP

Bài 3. Ba chú chó con – TRIDOGS

Bài 1. Cá mập cắn cáp – SHARK

Bài 2. RECTAREA

Bài 3. Tập lái xe – F1TRAINING

Bài 1. Cặp câu ý nghĩa – MEANING

Bài 2. Bắn bóng bay – BALLOONS

Bài 3. Rèm sân khấu – CURTAINS

Rèm cửa — CURTAINS

- ▶ Bình biểu diễn trên một sân khấu gồm n phần được đánh số từ 1 đến n từ trái sang phải. Anh ta còn có m tấm rèm được đánh số từ 1 đến m .
- ▶ Mỗi tấm trong m tấm rèm này có thể được hạ xuống. Rèm i hạ xuống sẽ che phủ các phần sân khấu từ $l[i]$ đến $r[i]$. Một cấu hình rèm là một tập các tấm rèm được hạ xuống. Cho một cấu hình rèm, phần sân khấu x ($1 \leq x \leq n$) được gọi là che phủ khi và chỉ khi tồn tại một tấm rèm i hạ xuống sao cho $l[i] \leq x \leq r[i]$.
- ▶ Bình muốn đưa ra tổng số q màn trình diễn, được đánh số từ 1 đến q . Đối với mỗi màn trình diễn j , Bình yêu cầu cấu hình rèm sao cho các phần sân khấu từ $s[j]$ đến $e[j]$ được che phủ và không che phần nào khác.
- ▶ **Yêu cầu:** Với mỗi màn trình diễn trong số q màn trình diễn này, hãy giúp Bình xác định xem có tồn tại cấu hình rèm thỏa mãn yêu cầu của anh ấy hay không.

Subtask 1 (10%): $1 \leq n, m, q \leq 200$: $O(n * m * q)$

- ▶ Với mỗi truy vấn i , lặp qua tất cả m tấm rèm. Một rèm j có thể được sử dụng nếu $s[i] \leq l[j]$ và $r[j] \leq e[i]$; tức là bức màn nằm hoàn toàn trong phạm vi truy vấn.
- ▶ Trong số tối đa m rèm có thể được sử dụng để trả lời truy vấn i , hãy đánh dấu riêng từng phần từ $l[j]$ đến $r[j]$ là bị che phủ, sau đó kiểm tra xem mọi phần từ $l[j]$ đến $r[j]$ có bị che không để trả lời truy vấn.

Subtask 2 (10%): $1 \leq n, m, q \leq 2000$

- ▶ Thực hiện cùng thao tác lặp để tìm các rêm có thể được sử dụng cho truy vấn i như trong subtask 1. Có nhiều cách để tăng tốc độ này.
- ▶ Cách dễ nhất là sử dụng bitset để thực hiện trong $O(nm/64)$, cho DPT toàn bộ là $O(nmq/64)$.
- ▶ Cũng có thể thực hiện trong $O(m \log m)$ bằng cách lưu trữ các phần được bao phủ dưới dạng tập hợp các phạm vi có thứ tự. Điều này dẫn đến DPT toàn bộ là $O(qm \log m)$.

Subtask 3 (15%): $1 \leq n \leq 2000$: $O(n^2 * q)$

- ▶ Nhận xét: có nhiều nhất 2000 phần nghĩa là chỉ có tối đa 2000^2 phạm vi truy vấn. Như vậy, ta có thể chuẩn bị một số loại tính toán trước cho tất cả các truy vấn có thể có.
- ▶ Có thể thực hiện điều này trong $O(n^2)$ bằng QHD. Đặt $dp[l][r]$ bằng 1 nếu một tập con các rèm có thể được chọn để chỉ che các phần $[l, r]$ và bằng 0 nếu trái lại.
- ▶ Khởi tạo: đánh dấu $dp[l][r] = 1$ nếu tồn tại một tấm màn che chính xác $[l, r]$ và 0 nếu trái lại. Bây giờ ta cập nhật các giá trị của $dp[.][.]$ theo thứ tự từ điển (tức là $(1, 1), (1, 2), \dots, (1, n), (2, 1), \dots, (n, n)$).

Subtask 3 (15%): $1 \leq n \leq 2000$: $O(n^2 * q)$

- ▶ Trong quá trình thực hiện, ta lưu trữ một biến j khác đại diện cho “giá trị lớn nhất $i < r$ sao cho $dp[l][i] = 1$ ”. Nếu $dp[l][r] = 1$ thì ta cập nhật i thành r , trái lại $dp[l][r]$ không thay đổi.
- ▶ Để xác định xem $dp[l][r] = 1$,
 - ▶ Nếu có một tấm rèm che $[l, r)$, rõ ràng $dp[l][r] = 1$.
 - ▶ Trái lại, phải tồn tại một số $j < r$ sao cho $dp[l][j] = 1$ và tồn tại một số i giữa l và j sao cho có một tấm rèm che chính xác $[i, r)$. Ta không cần kiểm tra tất cả j sao cho $dp[l][j] = 1$, mà thay vào đó, kiểm tra giá trị $j < r$ lớn nhất với $dp[l][j] = 1$ là đủ. Ta chỉ cần kiểm tra sự tồn tại i giữa l và j sao cho $dp[i][l] = 1$. Có thể sử dụng một mảng tổng tiền tố $p[i][l] = p[1][l] + \dots + p[i][l]$ để xử lý.

Subtask 4 (20%): $s[j] = 1$: $O(n + m + q)$

- ▶ Ta có thể thực hiện quét tương tự như subtask 3. Do $s[j] = 1$, ta có thể đặt $ans[i]$ bằng 1 nếu các tấm rèm có thể che chính xác $[1, i]$ và bằng 0 nếu trái lại.
- ▶ Ban đầu, đối với các rèm có $l[i] = 1$, ta có thể đặt ngay $ans[r[i]]$ thành 1. Ta cũng muốn duy trì cho mỗi điểm cuối bên phải R , giá trị L nhỏ nhất sao cho một rèm $[L, R]$ tồn tại hoặc không có rèm nào kết thúc tại R .
- ▶ Sau đó, quét i từ 1 đến n , duy trì mức j tối đa hiện tại với $j < i$ và $ans[j] = 1$.
- ▶ Sử dụng các kết quả tính toán trước đó, lấy L nhỏ nhất sao cho tồn tại một rèm $[L, i]$. Nếu $L - 1 \leq j$ thì $ans[i] = 1$, trái lại bằng 0.
- ▶ Để trả lời các truy vấn, chỉ cần kiểm tra xem $ans[s[j]] = 1$ hay không.

Subtask 5 (25%): $1 \leq n \leq 100000$: $O((n + m + q) \log^2 n)$

- ▶ Giới hạn cho thấy ĐPT sẽ tương tự $O(n\sqrt{n})$ hoặc $O(n \log^2 n)$.
- ▶ Có thể áp dụng các giải pháp sử dụng các nhóm có kích thước $\sqrt{n}$ hoặc chia để trị. Dưới đây là giải pháp $O(n \log^2 n)$.
- ▶ Xét một khoảng $[a, b]$ và tất cả các rèm có điểm cuối bên trái nằm bên trong $[a, b]$, ký hiệu là $S_{[a,b]}$ và sắp xếp không giảm theo điểm cuối bên phải. Giả sử truy vấn $[s, e]$ bao trùm toàn bộ $[a, b]$. Khi đó, các rèm trong $S_{[a,b]}$ mà ta có thể sử dụng sẽ tạo thành tiền tố của $S_{[a,b]}$ vì ràng buộc điểm cuối bên trái luôn nằm trong $[s, e]$. Gọi là $S_{[a,b]}(e)$.
- ▶ Tập các rèm sử dụng này có thể không che hết $[a, b]$ và cũng có thể che một số phần bên ngoài $[a, b]$.

Subtask 5 (25%): $1 \leq n \leq 100000$: $O((n + m + q) \log^2 n)$

- ▶ Nhận thấy không có tấm rèm nào như vậy có thể che được phần $x < a$, và nếu nó che được một phần phần $y > b$ thì tất cả các phần trong $[b, y]$ cũng được che phủ. Vì vậy, ta có thể liên kết mỗi tiền tố của các tấm rèm trong $S_{[a,b]}$ với:
 - ▶ Chỉ số x lớn nhất trong $[a, b]$ sao cho không có rèm nào ở tiền tố che x . (1)
 - ▶ Chỉ số y lớn nhất $> b$ sao cho có rèm ở tiền tố che phủ y . (2)
- ▶ Bây giờ hãy xem xét hai khoảng $[a, b]$ và $[b + 1, c]$ cùng với truy vấn $[s, e]$ bao trùm toàn bộ $[a, c]$. Nếu $S_{[a,b]}(e)$ chứa (1) thì $S_{[b+1,c]}(e)$ sẽ không bao giờ có thể che được nó. Mặt khác, nếu $S_{[b+1,c]}(e)$ chứa (1) thì $S_{[a,b]}(e)$ có thể che được nó tùy thuộc vào (2) của $S_{[a,b]}(e)$ đủ lớn.
- ▶ Ngoài ra, xét (2) của $S_{[a,b]}(e) \cup S_{[b+1,c]}(e)$ mở rộng: khi biết các (1) và (2) của hai khoảng liên kề, ta có thể xác định xem liệu hợp của chúng có chứa một (1) và (2) của chúng hay không. Nếu ta có thể chia phạm vi truy vấn thành một số lượng nhỏ các khoảng che phủ phạm vi, thì ta có thể xác định một cách hiệu quả xem liệu phần hợp cuối cùng của các rèm có chứa một (1) hay không.

Subtask 5 (25%): $1 \leq n \leq 100000$: $O((n + m + q) \log^2 n)$

- ▶ Ta cần xây dựng một cây phân đoạn trong đó mỗi nút trong cây phân đoạn lưu trữ các rēm có điểm cuối bên trái nằm trong phạm vi được biểu thị bởi nút đó, được sắp xếp không giảm theo điểm cuối bên phải. Sau đó ta tính toán (1) và (2) của từng tiền tố. Điều này có thể được thực hiện một cách hiệu quả bằng cách duy trì một DSU cho trong mỗi khoảng để kiểm tra phần nào chưa được đề cập, hoặc bằng cách sử dụng ý tưởng sắp xếp trộn để hợp nhất các kết quả từ các nút con với nút cha.
- ▶ Khi thực hiện một truy vấn, ta chia phạm vi truy vấn thành tối đa $2 \log n$ khoảng bằng cách sử dụng cây phân đoạn. Sau đó, ta sử dụng tìm kiếm nhị phân trên mỗi nút để xác định lỗ chính xác và giá trị tầm với cho từng phân đoạn. Sau đó ta có thể xử lý các phân đoạn từ trái sang phải. Nếu không có (1) nào tồn tại trong phạm vi truy vấn ở cuối thì câu trả lời là có. Trái lại, câu trả lời là không.
- ▶ Việc xây dựng cây phân đoạn nằm giữa $O((n + m) \log n)$ và $O((n + m) \log^2 n)$ tùy thuộc vào việc cài đặt. Mỗi truy vấn bao gồm $O(\log n)$ tìm kiếm nhị phân, mỗi lần tốn $O(\log n)$, do đó tổng số truy vấn là $O(q \log^2 n)$.

Subtask 6 (20%): $1 \leq n, m, q \leq 500000$:

$$O((n + m + q) \log n)$$

- ▶ Chia để trị (một cách khác sử dụng cây phân đoạn lazy)
- ▶ Thiết lập hàm đệ quy $dnc(X, Y)$. Đặt $m = \lfloor (X + Y)/2 \rfloor$. Trong quá trình $dnc(X, Y)$, ta sẽ xử lý tất cả các truy vấn j thỏa mãn $s[j] \leq m$ và $e[j] \geq m + 1$. Sau đó, ta sẽ gọi $dnc(X, m)$ và $dnc(m + 1, Y)$ để xử lý tất cả các truy vấn khác không vượt qua m .
- ▶ Bây giờ ta sẽ mô tả cách xử lý tất cả các truy vấn thỏa mãn $s[j] \leq m$ và $e[j] \geq m + 1$. Ta sẽ xác định $closeL(i)$ cho $X \leq i \leq m$ là j nhỏ nhất trong đó $j \geq m$, sao cho có thể che phủ $[i, j]$ một cách chính xác. Tương tự, $closeR(j)$ với $m + 1 \leq i \leq Y$ là i lớn nhất trong đó $i \leq m + 1$ sao cho có thể bao phủ chính xác $[i, j]$. Phạm vi truy vấn $[s, e]$ là tốt khi và chỉ khi $closeL(s) \leq e$ và $closeR(e) \geq s$.

Subtask 6 (20%): $1 \leq n, m, q \leq 500000$:

$$O((n + m + q) \log n)$$

- ▶ Bây giờ ta sẽ mô tả cách tính $closeL$. Khi tính toán $closeL(i)$, hãy xem xét tất cả các rềm có $l = i$. Trong số đó, chỉ có hai điều là quan trọng, một là r nhỏ nhất $\geq m$, và một là r lớn nhất $\leq m - 1$. Đối với trường hợp trước, đó chính là giá trị tiềm năng của $closeL(i)$.
- ▶ Đối với trường hợp thứ hai, ta sẽ cần tìm giá trị tối thiểu của $closeL(k)$ cho $i + 1 \leq k \leq r + 1$. Ta có thể làm như vậy bằng cách duy trì một ngăn xếp đơn lưu trữ các cặp $(k, closeL(k))$. Ta đưa ra tất cả các giá trị trong ngăn xếp có $k \leq r + 1$ và đặt giá trị tối thiểu là $closeL(i)$ cho giá trị đó. Sau đó, ta sẽ biết giá trị nhỏ nhất của $closeL(i)$. Sau đó ta có thể đẩy nó vào ngăn xếp đơn.
- ▶ Tính toán $closeR$ tương tự.

Bức tường gạch — BRICKWALL

Những viên gạch có hình dạng là một hình vuông kích thước 1×1 . Ở đầu cổng làng, người ta thấy một bức tường gạch lớn được xây dựng từ N bức tường nhỏ sát cạnh nhau. Các bức tường được đánh số lần lượt từ 1 đến N . Bức tường gạch nhỏ thứ i có dạng hình chữ nhật chiều cao là h_i , chiều dài là w_i , được xây dựng bởi $h_i \times w_i$ viên gạch.

Ví dụ: Có 2 bức tường với $(h_1, w_1) = (1, 1)$ và $(h_2, w_2) = (2, 2)$ thì hình dạng của bức tường lớn được minh họa như sau:



Nhìn vào bức tường lớn đó, người ta thắc mắc một vấn đề rằng có bao nhiêu tập các viên gạch khác nhau thỏa mãn: Tập các viên gạch đó trên bức tường lớn xếp thành một hình chữ nhật. Hãy xác định số lượng tập các viên gạch thỏa mãn điều trên.

Subtask 1 (15%): $N \leq 50, h_i \leq 50, w_i = 1$ với mọi i :
 $O(N^4)$

Tạo 1 bảng hình chữ nhật kích thước $50 * 50$, để mô tả bức tường lớn. Ta biểu diễn các ô có chứa viên gạch là số 1. Số lượng tập gạch thỏa mãn bằng với số lượng hình chữ nhật con gồm toàn số 1 có trong bảng. Sử dụng quy hoạch động để đếm.

Subtask 2 (15%): $h_i = 1$ hoặc $h_i = 2$ với mọi i : $O(N)$

- ▶ Ta coi như một tập gạch thỏa mãn là một hình chữ nhật.
- ▶ Dễ thấy các hình chữ nhật chỉ gồm 2 loại với chiều cao là 1 và 2. Ta xem bức tường được chia thành 2 hàng, tính từ dưới lên trên là hàng 1 và hàng 2.
- ▶ Đặt $S = w_1 + w_2 + \dots + w_N$, ta có số lượng hình chữ nhật chiều cao 1 trong hàng 1 là $C(2, S + 1)$.
- ▶ Xét các bức tường từ l đến r có chiều cao là 2. Đặt $K = w_l + w_{l+1} + \dots + w_r$, ta có số lượng hình chữ nhật chiều cao 2 là $C(2, K + 1)$ và số lượng hình chữ nhật chiều cao bằng 1 ở hàng 2 cũng là $C(2, K + 1)$.

Subtask 3 (15%): $h_1 = h_2 = \dots = h_N: O(N)$

Đặt $H = h_1 = h_2 = \dots = h_N$, $S = w_1 + w_2 + \dots + w_N$.

Số lượng hình chữ nhật thỏa mãn là $C(2, K + 1) * C(2, S + 1)$.

Subtask 4 (15%): $h_i \leq h_{i+1}$ với $i \leq N - 1$: $O(N)$

Xét một hình chữ nhật lớn có kích thước chiều cao A và chiều rộng là B . Gọi số lượng hình chữ nhật con có trong này là $T_{A,B}$.

$$T_{A,B} = C(2, A + 1) * C(2, B + 1).$$

Đặt $S_i = w_i + w_{i+1} + \dots + w_N$.

Kết quả là: $\sum_{i=1}^N T_{h_i, S_i} - T_{h_{i-1}, S_i}$.

Subtask 5 (20%): $N \leq 1000$: $O(N^2)$

Gọi H là chiều cao hình chữ nhật thấp nhất trong các hình chữ nhật từ i đến j .

Số lượng hình chữ nhật con có phần cạnh bên trái thuộc hình chữ nhật thứ i và cạnh bên phải thuộc hình chữ nhật thứ j là:

- ▶ $cnt[i, i] = C(2, h_i + 1) * C(2, w_i + 1)$.
- ▶ $cnt[i, j] = C(2, H + 1) * w_i * w_j$ ($i < j$).

Kết quả bài toán: $Ans = \sum_{i=1}^N \sum_{j=i}^N cnt[i, j]$.

Subtask 6 (20%): $N \leq 10^5$: $O(N * \log N)$

Sử dụng DSU để gộp các hình chữ nhật liên tiếp cạnh nhau.

- ▶ $root(i)$ là chỉ số đại diện của tập hình chữ nhật có chứa hình chữ nhật thứ i .
- ▶ $W[root(i)]$ là tổng chiều rộng các hình như nhật có trong tập.

Subtask 6 (20%): $N \leq 10^5$: $O(N * \log N)$

Ta sử dụng DSU để gộp các hình chữ nhật liên tiếp cạnh nhau.

- ▶ $root(i)$ là chỉ số đại diện của tập hình chữ nhật có chứa hình chữ nhật thứ i .
- ▶ $W[root(i)]$ là tổng chiều rộng các hình như nhật có trong tập.

Ta lần lượt xét các cột theo thứ tự chiều cao giảm dần:

- ▶ Khi xét đến hình chữ nhật thứ i có chiều cao là h_i , nếu như hình chữ nhật thứ $i - 1$ có $h_{i-1} \geq h_i$ (hình chữ nhật $i - 1$ đã được xét trước) thì ta tiến hành join hình chữ nhật thứ i vào tập $root(i - 1)$. Vì h_i là chiều cao bé nhất trong tập nên ta sẽ đếm số lượng hình chữ nhật con có:

- ▶ Cạnh bên trái thuộc trong tập $root(i - 1)$.
- ▶ Cạnh bên phải thuộc hình chữ nhật thứ i .
- ▶ Số hình chữ nhật con thỏa mãn là:

$$Ans+ = T_{h_i, W[root(i)]+w_i} - T_{h_i, W[root(i)]} - T_{h_i, w_i}.$$

Subtask 6 (20%): $N \leq 10^5$: $O(N * \log N)$

- ▶ Khi join hình chữ nhật thứ i vào tập $root(i - 1)$ thì:
 - ▶ $root(i) = root(i - 1)$.
 - ▶ $W[root(i)]_+ = w_i$.
- ▶ Tương tự, nếu hình chữ nhật thứ $i + 1$ có $h_{i+1} \geq h_i$ (hình chữ nhật thứ $i + 1$ đã được xét trước), thì ta cũng làm như trên.

Dãy xâu con đối xứng – PALINSUB

Phong đưa bạn một dãy a chứa các xâu chỉ bao gồm các chữ cái in thường. Nhiệm vụ của bạn là tìm xem có bao nhiêu dãy con các xâu là một dãy palindrome. Một dãy các xâu được gọi là dãy palindrome nếu xâu được tạo thành bằng cách ghép các xâu lại theo thứ tự tạo thành một xâu đối xứng.

Subtask 1 (40 %): $n \leq 10$: $O(2^N * \sum |a_i|)$

Sinh ra tất cả xâu có thể tạo được rồi kiểm tra.

Subtask 2 (30 %): độ dài tất cả các xâu đều bằng 1: $O(N^2)$

Dùng QHĐ, gọi $dp[x][y]$ là số xâu con đối xứng có thể tạo được trong đoạn $a_x, a_{x+1}, \dots, a_y$. Nhận xét:

- ▶ Nếu $a_x = a_y$, mọi xâu đối xứng trong $[x+1, y-1]$ (kể cả xâu rỗng) đều có thể được mở rộng bằng cách thêm a_x và $a_y \Rightarrow dp[x+1][y-1]$
- ▶ Nếu không chọn mở rộng các xâu còn lại được chọn ở một trong hai đoạn $[x+1, y]$ và $[x, y-1]$ hoặc cả hai. Các xâu được tính hai lần nằm trong đoạn
$$[x+1][y-1] \Rightarrow dp[x][y-1] + dp[x+1][y] - dp[x+1][y-1]$$
$$\Rightarrow dp[x][y] =$$

$$dp[x+1][y-1] * (a[x] == a[y]) + dp[x][y-1] + dp[x+1][y] - dp[x+1][y-1]$$

$$ans = dp[1][n]$$

Cơ sở: $dp[x][x] = 2$ (tính cả xâu rỗng) và $dp[x][y] = 1$ với mọi $x > y$ (xâu rỗng).

Subtask 3 (30 %): $n \leq 3000$: $O((\sum |a_i|)^2)$

- ▶ Sử dụng ý tưởng tương tự subtask 2, tuy nhiên mỗi phần tử bây giờ là một xâu thay vì một ký tự.
- ▶ Vấn đề ở subtask này là khi ta chọn $a[x]$ và $a[y]$, độ dài của chúng có thể không bằng nhau. Do đó, khi thực hiện so khớp lần lượt, một bên đã hết nhưng bên kia vẫn còn ký tự cần so khớp. Ta cần lưu lại phần còn dư này để thực hiện so khớp tiếp với xâu tiếp theo.
- ▶ Để đơn giản, ta nối các xâu lại với nhau, thêm ký tự '|' đầu, cuối và ở giữa các xâu để biết bắt đầu/kết thúc của một xâu. Gọi xâu vừa tạo được là s .

Subtask 3 (30 %): $n \leq 3000$: $O((\sum |a_i|)^2)$

Cơ sở: tùy thuộc vào cách cài đặt, có một số lưu ý như sau:

- ▶ Ta chỉ tính sâu rộng vào kết quả khi và chỉ khi $s[x] = s[y] = \text{'|'}$.
- ▶ $x > y$ có thể ứng với nhiều trường hợp khác nhau, ví dụ trạng thái xuất phát từ trường hợp thứ 4 khác trạng thái xuất phát từ trường hợp thứ 2 hoặc 3 trong công thức chuyển trạng thái sau.

Subtask 3 (30 %): $n \leq 3000$: $O((\sum |a_i|)^2)$

Chuyển trạng thái cho $dp[x][y]$:

- ▶ $s[x]$ và $s[y]$ đều là chữ cái, ta tiếp tục so khớp:
 $dp[x][y] = dp[x+1][y-1]$ nếu $s[x] = s[y]$, ngược lại
 $dp[x][y] = 0$.
- ▶ $s[x]$ là chữ cái, $s[y] = '|'$, bên phải đã hết nhưng bên trái vẫn còn. Ta cần chọn một xâu tiếp theo cho bên phải (nếu dùng for sẽ bị quá thời gian) bằng cách chọn một trong hai thao tác: bắt đầu so khớp xâu tiếp theo bên phải hoặc bỏ qua nó.
 $\Rightarrow dp[x][y-1] + dp[x][nxt]$ với nxt là vị trí '|' tiếp theo bên trái của y .
- ▶ $s[y]$ là chữ cái, $s[x] = '|'$. Tương tự trường hợp trên.
- ▶ $s[x] = s[y] = '|'$ tương tự subtask trên, ta có hai cách: chọn cả $s[x]$ và $s[y]$ để mở rộng hoặc không chọn. $\Rightarrow$
 $dp[x+1][y-1] + (dp[nxtX][y] + dp[x][nxtY] - dp[nxtX][nxtY])$
với $nxtX$ ($nxtY$) là vị trí '|' tiếp theo bên phải (trái) của x (y).

Mạng liên kết — NETWORK

- ▶ Mạng bao gồm n máy chủ và $n - 1$ đường liên kết mạng kết nối các máy chủ này liên thông.
- ▶ Ban đầu, tất cả n máy chủ đều đang hoạt động. Đắc có m ứng dụng đang chạy, có ID riêng biệt từ 1 đến m . Mỗi ứng dụng có 2 cơ sở dữ liệu. Ứng dụng j có cơ sở dữ liệu trong máy chủ $a[j]$ và $b[j]$ (có thể là cùng một máy chủ). Hai ứng dụng khác nhau có thể có cơ sở dữ liệu trên cùng một máy chủ.
- ▶ Để ứng dụng j hoạt động, cả hai cơ sở dữ liệu của nó phải được kết nối thông qua các liên kết mạng và máy chủ đang hoạt động.
- ▶ Bình có thể chọn bất kỳ bộ máy chủ nào và vô hiệu hóa tất cả chúng cùng một lúc. Độ bền của mạng là số lượng máy chủ tối thiểu phải ngừng hoạt động để đảm bảo tất cả m ứng dụng không hoạt động.
- ▶ **Yêu cầu:** Tính toán độ bền của mạng.

Subtask 1 (4%): $a[i] = b[l]$: $O(n + m)$

- ▶ Mỗi ứng dụng có cả 2 cơ sở dữ liệu của nó trong cùng một máy chủ. Điều kiện rút gọn thành: "ứng dụng j đang hoạt động khi và chỉ khi máy chủ $a[j]$ hoạt động.
- ▶ Để đảm bảo rằng tất cả các ứng dụng không hoạt động, tất cả các máy chủ $a[j]$ phải được chọn để ngừng hoạt động.
- ▶ Việc vô hiệu hóa các máy chủ còn lại sẽ không ảnh hưởng đến việc ứng dụng có hoạt động hay không, nhưng ta muốn giảm thiểu số lượng máy chủ bị vô hiệu hóa nên không được chọn chúng. Do đó, tập hợp các máy chủ được chỉ định ngừng hoạt động chính xác là tập hợp các máy chủ $a[j]$ xuất hiện trong input.
- ▶ Kết quả bằng số lượng các giá trị xuất hiện duy nhất trong mảng a .

Subtask 2 (17%): $u[l] = i, v[i] = i + 1: O(n + m \log m)$

- ▶ Các điều kiện để ứng dụng j hoạt động không thay đổi khi $a[j]$ và $b[j]$ được đổi chỗ. Do đó, ta giả thiết $a[j] \leq b[j]$. Nếu dữ liệu ngược lại, hoán đổi $a[j]$ và $b[j]$.
- ▶ Thuật toán: Sắp xếp các ứng dụng không giảm theo $a[j]$. Duy trì một tập S , ban đầu rỗng. Đối với mỗi ứng dụng j :
 - ▶ Gọi x là số nhỏ nhất trong S . Nếu $a[j] > x$, hủy kích hoạt x và làm rỗng S .
 - ▶ Cộng $b[j]$ vào S .

Cuối cùng, hủy kích hoạt ứng dụng tương ứng với số nhỏ nhất trong S . (Chúng ta có thể tăng tốc độ bước này bằng cách lưu số nhỏ nhất trong S thay vì toàn bộ tập hợp S .)

Subtask 2 (17%): $u[l] = i, v[i] = i + 1: O(n + m \log m)$

- ▶ Đầu tiên, ta chứng minh thuật toán này vô hiệu hóa tất cả các ứng dụng: Quan sát khi chúng ta vô hiệu hóa x , với mỗi j' tương ứng với các phần tử trong S , ta phải có $b[j'] \geq x$ (vì x là nhỏ nhất). Ngoài ra, ta có $a[j'] \leq x$, nếu không S sẽ bị làm rỗng trước đó.
- ▶ Để chỉ ra thuật toán tối ưu, hãy quan sát mỗi khi ta làm rỗng S , x được tạo ra bởi một ứng dụng nào đó $(a[j], b[j])$. Có thể chỉ ra rằng tập hợp các $\{(a_j, b_j)\}$ này tương ứng với các khoảng rời rạc và do đó mỗi ứng dụng trong số chúng phải bị một máy chủ khác nhau vô hiệu hóa. Vì vậy đây là số lượng tối thiểu cần tìm.

Subtask 3 (14%): $n, m \leq 15$: Time: $O(2^n nm)$, Space: $O(nm)$

- Gọi $S[j]$ là tập hợp tất cả các đỉnh nằm trên đường đi từ $a[j]$ đến $b[j]$ với mỗi j . Lưu ý rằng nếu cây mạng có gốc ở đỉnh $a[j]$ thì đỉnh v nằm dọc theo đường đi khi và chỉ khi v là tổ tiên của $b[j]$. Do đó, ta có thể thực hiện DFS từ $a[j]$ để tìm cha trực tiếp của mỗi đỉnh, sau đó di chuyển liên tục lên từ đỉnh $b[j]$ để tìm tất cả các đỉnh dọc theo đường đi. Quá trình này chạy trong $O(n)$ cho mỗi ứng dụng, cho tổng ĐPT thời gian là $O(nm)$. Việc lưu trữ tất cả $S[j]$ cho ĐPT không gian là $O(nm)$.
- Lặp lại tất cả 2^n tập hợp con có thể có của các đỉnh để chọn C và kiểm tra xem mỗi C có thể khiến tất cả các ứng dụng không hoạt động hay không. Ứng dụng j không hoạt động khi và chỉ khi có một đỉnh trong cả C và $S[j]$. Việc kiểm tra này có thể được thực hiện trong thời gian $O(|C| * |S[j]|) = O(nm)$ cho mỗi tập hợp con.

Subtask 4 (29%): $n, m \leq 200$: $O(n^2 + nm + m \log m)$

- ▶ Ta bắt đầu bằng cách lấy gốc cây ở một đỉnh tùy ý và thực hiện DFS để tìm cha mẹ trực tiếp và độ sâu của mỗi đỉnh. Đối với mỗi ứng dụng j , trước tiên hãy tính tổ tiên chung thấp nhất của $a[j]$ và $b[j]$, gọi là $\ell[j]$. Chỉ cần áp dụng thuật toán tìm tổ tiên chung gần nhất trong thời gian $O(n)$.
- ▶ *Khẳng định*: Giả sử tồn tại một giá trị của j sao cho với mọi $j' \neq j$, $\ell[j']$ không phải là con cháu thực sự của $\ell[j]$, thì tồn tại một nghiệm tối ưu vô hiệu hóa $\ell[j]$.

Subtask 4 (29%): $n, m \leq 200$: $O(n^2 + nm + m \log m)$

- ▶ Chứng minh: Xét một lời giải và giả sử rằng ứng dụng j bị vô hiệu hóa bởi nút x (nếu có nhiều x như vậy, chọn bất kỳ). Ta hoàn tác (undo) việc hủy kích hoạt x và thay vào đó là hủy kích hoạt $\ell[j]$. Vẫn còn phải chứng minh cấu hình mới này sẽ vô hiệu hóa tất cả các máy chủ. CM phản chứng, giả sử tồn tại một số j' bị vô hiệu hóa bởi x , nhưng không bị vô hiệu hóa bởi $\ell[j]$. Vì x vô hiệu hóa j' , nên một trong $a[j']$ và $b[j']$ là hậu duệ của x , trái giả thiết phản chứng giả sử $a[j']$ là hậu duệ của x (cũng là hậu duệ của $\ell[j]$). Nếu $b[j']$ cũng là hậu duệ của ℓ thì $\ell[j']$ là tổ tiên chung của cả $a[j']$ và $b[j']$. Mặt khác, ta giả sử rằng $\ell[j']$ không phải là hậu duệ thực sự của $\ell[j]$, vì vậy $\ell[j'] = \ell[j]$, có nghĩa là đường đi duy nhất từ $a[j']$ đến $b[j']$ đi qua $\ell[j]$. Nếu $b[j']$ không phải là hậu duệ của ℓ thì đường đi duy nhất từ $a[j']$ đến $b[j']$ đi qua $\ell[j]$. Trong mọi trường hợp, $\ell[j]$ vô hiệu hóa j' .

Subtask 4 (29%): $n, m \leq 200$: $O(n^2 + nm + m \log m)$

- ▶ Điều này gợi ý thuật toán sau: đầu tiên tính toán $\ell[j]$ cho tất cả j và giữ một tập S gồm các ứng dụng được kích hoạt, được khởi tạo thành $\{1, 2, \dots, m\}$. Trong khi S không trống, hãy trích xuất một số j sao cho $\ell[j]$ có khoảng cách tối đa từ gốc và hủy kích hoạt $\ell[j]$. Khi điều này xảy ra, hãy tìm tất cả j' bị vô hiệu hóa bởi $\ell[j]$ và xóa chúng khỏi S .
- ▶ Lưu ý rằng ta có thể trì hoãn việc hủy kích hoạt ứng dụng j' . Thay vào đó, ta xử lý các ứng dụng bằng cách giảm độ sâu $\ell[j]$ và kiểm tra xem ứng dụng đó có bị vô hiệu hóa trước đó hay không. Để làm như vậy, hãy lặp lại tất cả các đỉnh giữa $a[j]$ và $b[j]$ và xem liệu có đỉnh nào bị vô hiệu hóa hay không. Việc sắp xếp các ứng dụng mất $O(m \log m)$ thời gian, trong khi việc kiểm tra xem một ứng dụng đã bị vô hiệu hóa hay không mất $O(n)$ thời gian cho mỗi ứng dụng.

Subtask 5 (36%): $1 \leq n, m \leq 200000$:

$O(n \log n + m \log n + m \log m)$

- ▶ Tăng tốc giải pháp từ Subtask 4.
- ▶ Bây giờ, ta phải tính tổ tiên chung thấp nhất trong thời gian $O(\log n)$ hoặc tốt hơn.
- ▶ Ta cũng cần đẩy nhanh quá trình kiểm tra xem ứng dụng đã bị vô hiệu hóa hay chưa. Lưu ý rằng đường đi từ $a[j]$ đến $b[j]$ có thể được chia thành hai đường đi từ $\ell[j]$ đến $a[j]$ và $b[j]$. Gọi $p[i]$ là số đỉnh bị vô hiệu hóa trên đường đi từ gốc đến đỉnh i và đặt $p[\text{parent}[\text{root}]] = 0$.
- ▶ Vì $\ell[j]$ là tổ tiên của $a[j]$, nên tồn tại một đỉnh bị vô hiệu hóa trên đường đi từ $\ell[j]$ đến $a[j]$ khi và chỉ khi $p[a[j]] > p[\text{parent}[\ell[j]]]$. Tương tự, chúng ta có thể kiểm tra xem có đỉnh nào bị vô hiệu hóa trên đường đi từ $\ell[j]$ đến $b[j]$.

Subtask 5 (36%): $1 \leq n, m \leq 200000$:

$O(n \log n + m \log n + m \log m)$

- Ta chỉ còn lại việc cập nhật và truy vấn nhanh chóng các giá trị của $p[i]$. Lưu ý rằng khi một đỉnh x bị vô hiệu hóa, $p[x']$ tăng thêm 1, với tất cả x' nằm trong cây con có gốc tại x . Do cây con gốc của các đỉnh có các nhãn liên tiếp theo thứ tự trước của cây, nên bằng cách chuyển đổi tất cả các nhãn đỉnh thành nhãn thứ tự trước của chúng, việc tăng $p[x']$ lên 1 sẽ trở thành một cập nhật khoảng. Bằng cách sử dụng cây Fenwick, các cập nhật khoảng cho $p[x']$ và các truy vấn điểm của $p[i]$ đều có thể được thực hiện trong thời gian $O(\log n)$.

Subtask 5 (36%): $1 \leq n, m \leq 200000$: Cách 2: Quy hoạch động trên cây

- ▶ Gọi $S[j]$ là tập hợp tất cả các đỉnh nằm trên đường đi từ $a[j]$ đến $b[j]$ với mỗi j .
- ▶ Ans là số đỉnh ít nhất xuất hiện trong tất cả các tập S . (Đồ thị G gồm mỗi tập $S[j]$ là một đỉnh đồ thị, 2 tập $S[i], S[j]$ có phần tử chung thì có một cạnh giữa 2 đỉnh tương ứng. Bài toán đưa về tìm tập phủ đỉnh nhỏ nhất (set cover) trên G . Bài toán đối ngẫu là tìm tập độc lập cực đại lớn nhất trên G).
- ▶ Đưa về bài toán quy hoạch động trên cây: Tính $f[u]$ là số lượng tập S cực đại của cây con gốc u :
 - ▶ nếu u không phải là LCA của $a[i], b[i]$ nào thì $f[u] = \sum f[v]$ với mọi v là con của u ;
 - ▶ nếu u là LCA của $a[i], b[i]$ nào đó: $f[u] = \sum f[v] + 1$ với mọi v là con của một đỉnh nằm trên $S[i]$.